

# Foundations of Artificial Intelligence

M. Helmert  
C. Büchner

University of Basel  
Spring Term 2026

## Exercise Sheet 2

Due: March 8, 2026

Please note the submission rules at the end of this exercise sheet. Non-adherence to these rules may lead to your submission not being marked.

### Exercise 2.1 (1+1 marks)

Characterize the following environments by describing if they are *static* / *dynamic*, *deterministic* / *non-deterministic* / *stochastic*, *fully* / *partially* / *not observable*, *discrete* / *continuous*, and *single-agent* / *multi-agent*. Explain your answers.

(a) Monopoly ([https://en.wikipedia.org/wiki/Monopoly\\_%28game%29](https://en.wikipedia.org/wiki/Monopoly_%28game%29))

(b) Mars rover ([https://en.wikipedia.org/wiki/Mars\\_rover](https://en.wikipedia.org/wiki/Mars_rover))

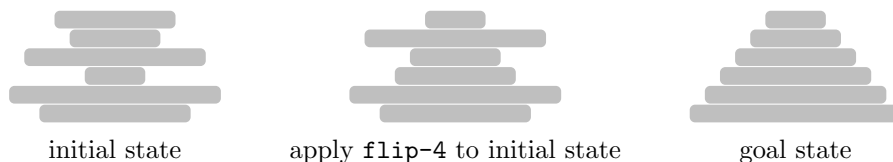
### Exercise 2.2 (0.5+0.5+0.5+0.5 marks)

Determine if the following statements are true for all state spaces  $\mathcal{S} = \langle S, A, cost, T, s_I, S_G \rangle$ . Explain your answers.

- (a) If all actions have cost 0, each solution for  $\mathcal{S}$  is optimal.
- (b) If  $\pi = \langle a_1, \dots, a_n \rangle$  is a minimal cost path from state  $s \in S$  to state  $s' \in S$  and  $\pi' = \langle a'_1, \dots, a'_m \rangle$  is a minimal cost path from  $s'$  to state  $s'' \in S$ , then  $\pi'' = \langle a_1, \dots, a_n, a'_1, \dots, a'_m \rangle$  is a minimal cost path from  $s$  to  $s''$ .
- (c) It is possible for unit-cost state spaces (where all actions have cost 1) that some state has the same optimal solution cost as one of its successors.
- (d) If there is no solution for some state, then there is no solution for any of its predecessors.

### Exercise 2.3 (2 marks)

In the *pancake* problem we have  $n$  pancakes with size  $1, \dots, n$  on a pile. Action **flip- $i$**  with cost 1 flips the top  $i$  pancakes. The goal is to order the pile by size, i.e. the largest pancake is on the bottom, the second largest on top of the largest and so on. The following figure shows an example instance:



Formalize the state space of the above pancake problem instance. Specify all parts of the state space, i.e. the set of states, the set of actions, the cost function, the set of transitions, the initial state and the set of goal states.

### Exercise 2.4 (3+1 marks)

The task in this exercise is to write a software program. We expect you to implement your code without using existing code you find online. If you encounter technical problems or have difficulties understanding the task, please let us know.

On the ADAM website of the course, you find Java code (`state-spaces.zip`) that implements the black box interface for state spaces that was discussed in the lecture, as well as an interface for states and actions. The code includes an example implementation of the interface for the blocks world problem. You can test the implementation by invoking the `StateSpaceTest` class, which creates a set of random successor states starting from the initial state. To run the program, first compile it with

```
javac StateSpaceTest.java
```

from a shell (on Linux) and then run it with

```
java StateSpaceTest blocks blocks-problem.txt
```

The sole purpose of the provided blocks world implementation is to serve as an example that helps you for your own implementation.

Your task is to implement the provided interface for the *pancakes* problem. Everything should be implemented in a single file called `PancakesStateSpace.java`. **In your handin, only submit your `PancakesStateSpace.java` as a solution to this exercise, any other files will be ignored.**

- (a) Implement the state space of the pancakes problem (for a variable number of pancakes).
- (b) Implement the `buildFromCmdline` function for the pancakes problem such that it parses an input file where the pancakes are listed by size from top to bottom, separated by whitespace. The file `pancakes-problem.txt` from the archive encodes the instance from Exercise 2.3. Make sure that your code recognizes invalid inputs such as missing or duplicate pancakes.

You can test your implementation by uncommenting the two indicated lines in the method `createStateSpace` of the `StateSpaceTest` class and then executing the command

```
java StateSpaceTest pancakes pancakes-problem.txt
```

Notice that the command uses the new keyword `pancakes` (rather than `blocks` as in the example above). Also notice that the problem is not expected to be solved by the provided random walk.

#### Submission rules:

- Submit one solution in groups of up to two students, stating the names of all group members in the submission. All group members are equally responsible for all submitted exercises.
- Create a single A4 PDF file for all non-programming exercises. Solutions in  $\text{\LaTeX}$  are preferred, but clearly legible scans in A4 with reasonable contrast and file size are accepted as well.
- In case there are programming exercises, upload a ZIP file (ending `.zip`, `.tar.gz` or `.tgz`; *not* `.rar` or anything else) including your PDF and those code textfiles required by the exercise. Put your names in a comment on top of each code file. Make sure your code compiles and test it. Code that does not compile or which we cannot successfully execute will not be graded.
- Include a statement on scientific integrity at the end of your submission (see below).

#### Statement on scientific integrity:

- All submitted solutions must be the product of your own work.
- If your written solutions were produced using any help (e.g., from peers or a large language model), list in what way these people or tools contributed to the solutions.

- We do not mark submissions that lack a statement on scientific integrity.
- Examples:
  - “We solved all exercises on our own.”
  - “We used ChatGPT to explain exercise 7 which we did not understand.”
  - “We had no time to solve this exercise sheet and copied all solutions from our friends.”  
*(If this is the case, thanks for your honesty but this is plagiarism, results in a warning and no marks. It is preferable to submit no solution at all.)*